

Approaches to Styling React Frontends

In this part, we will style the alert component we worked on in previous part using four different approaches. First, we will use plain CSS and understand the downsides of this approach. Then, we will move on to use **CSS modules**, which will resolve plain CSS's main problem. We will then use a **CSS-in-JS** library called Emotion and a library called Tailwind CSS and will understand the benefits of each of these libraries.

We will also learn how to use SVGs in React apps and use them in the alert component for the information and warning icons.

We'll cover the following topics:

- Using plain CSS
- Using CSS modules
- Using CSS-in-JS
- Using Tailwind CSS
- Using SVGs

Technical requirements

We will use the following technologies in this part:

- **Browser:** A modern browser such as Google Chrome
- **Node.js** and **npm:** You can install them from <https://nodejs.org/en/download/>
- **Visual Studio Code:** You can install it from <https://code.visualstudio.com/>

Using plain CSS

We will start this section by setting up a React and TypeScript project with the alert component from previous part, *Setting Up React and TypeScript*. Next, we will add the alert component from previous part and style it using plain CSS. Finally, we will look at one of the challenges with plain CSS and discover how we could mitigate it.

Creating the project

The project is now set up. Next, we will take some time to understand how to use plain CSS in React components.

Understanding how to reference CSS

Create React App has already enabled the use of plain CSS in the project. In fact, if you look in **App.tsx**, it already uses plain CSS:

```

...
import './App.css';
...
function App() {
  return (
    <div className="App">
      ...
    </div>
  );
}
...

```

CSS styles from the **App.css** file are imported, and the **App** CSS class is referenced on the outer **div** element.

React uses a **className** attribute rather than **class** because **class** is a reserved word in JavaScript. The **className** attribute is converted to a **class** attribute during transpilation.

The CSS import statement is a webpack feature. As webpack processes all the files, it will include all the imported CSS in the bundle.

Carry out the following steps to explore the CSS bundle that the project produces:

1. Start by opening and looking at **App.css**. As we have seen already, **App.css** is used within **App.tsx**. However, it contains CSS classes that are no longer used, such as **App-header** and **App-logo**. These classes were referenced in the **App** component before we removed them when we added the alert component. Leave the redundant CSS classes in place.
2. Open the **index.tsx** file and you'll notice that **index.css** is imported. However, no CSS classes are referenced within this file. If you open **index.css**, you will notice that it only contains CSS rules that target element names and no CSS classes.
3. Run the following command in the terminal to produce a production build:

```
npm run build
```

After a few seconds, the build artifacts will appear in a **build** folder at the project's root.

4. Open **index.html** in the **build** folder and notice all the whitespace has been removed because it is optimized for production. Next, find the **link** element that references the CSS file and note down the path – it will be something similar to **/static/css/main.073c9b0a.css**.



Figure 5.1 – The link element in index.html

5. Open up the referenced CSS file. All the whitespace has been removed because it is optimized for production. Notice that it contains all the CSS from **index.css** and **App.css**, including the redundant **App-header** and **App-logo** CSS classes.



Figure 5.2 – The bundled CSS file, including redundant App-header CSS class

The key point here is that webpack doesn't remove any redundant CSS – it will include all the content from all the CSS files that have been imported.

Next, we will style the alert component with plain CSS.

Using plain CSS in the alert component

Now that we understand how to use plain CSS within React, let's style the alert component. Carry out the following steps:

1. Add a CSS file called **Alert.css** in the **src** folder to copy.
2. We will add the CSS classes step by step and understand the styles in each class. Start by adding a **container** class into **Alert.css**:

```
.container {

  display: inline-flex;

  flex-direction: column;

  text-align: left;

  padding: 10px 15px;

  border-radius: 4px;

  border: 1px solid transparent;

}
```

This will be used on the outer **div** element. The style uses an inline flexbox, with the items flowing vertically and left-aligned. We've also added a nice rounded border and a bit of padding between the border and child elements.

3. Add the following additional classes that can be used within **container**:

```
.container.warning {  
  
    color: #e7650f;  
  
    background-color: #f3e8da;  
  
}
```

```
.container.information {  
  
    color: #118da0;  
  
    background-color: #dcf1f3;  
  
}
```

We will use these classes for the different types of alerts to color them appropriately.

4. Add the following class for the header container element:

```
.header {  
  
    display: flex;  
  
    align-items: center;
```

```
margin-bottom: 5px;
```

```
}
```

This will be applied to the element that contains the icon, heading, and close button. It uses a flexbox that flows horizontally with child elements vertically centered. It also adds a small gap at the bottom before the alert message.

5. Now add the following class for the icon to give it a width of 30 px:

```
.header-icon {
```

```
width: 30px;
```

```
}
```

6. Next, add the following class to apply to the heading to make it bold:

```
.header-text {
```

```
font-weight: bold;
```

```
}
```

7. Add the following class to apply to the close button:

```
.close-button {
```

```
border: none;
```

```
background: transparent;
```

```
margin-left: auto;
```

```
cursor: pointer;
```

```
}
```

This removes the border and background. It also aligns the button to the right of the header and gives it a pointer mouse cursor.

8. Add the following class for the content element:

```
.content {  
  
    margin-left: 30px;  
  
    color: #000;  
  
}
```

This adds a left margin so the message horizontally aligns with the heading and sets the text color to black.

That completes all the CSS class definitions.

9. Open **Alert.tsx** and add an import statement for the CSS file we just created:

```
import './Alert.css';
```

10. Now we are going to reference the CSS classes we just created in the elements of the alert component. Add the following highlighted CSS class name references in the alert JSX to do this:

```
<div className={`container ${type}`}>
```

```
<div className="header">
```

```
<span
```

```
...
```

```
    className="header-icon"
```

>

{type === "warning" ? "⚠" : "!"}

{heading}

</div>

{closable && (

<button

...

className="close-button"

>

...

</button>

)}

<div className="content">{children}</div>

```
</div>
```

The elements in the alert component are now being styled by the CSS classes in the imported CSS file.

11. Move the close button so that it is located inside the header container, under the **header** element:

```
<div className={`container ${type}`}>
```

```
<div className="header">
```

```
...
```

```
<span className="header-text">{heading}</span>
```

```
{closable && (
```

```
<button
```

```
aria-label="Close"
```

```
onClick={handleCloseClick}
```

```
className="close-button"
```

```
>
```

```
<span role="img" aria-label="Close">
```




```
</span>
```

```
</button>
```

```
}}
```

```
</div>
```

```
<div className="content">{children}</div>
```

```
</div>;
```

12. Start the app in development mode by running **npm start** in the terminal.

After a few seconds an improved alert component will appear in the browser:

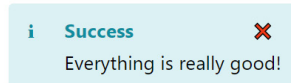


Figure 5.3 – A styled alert component with plain CSS

That completes the alert component's styling, but let's continue so that we can observe a downside of plain CSS.

Experiencing CSS clashes

We will now see an example of CSS with different components clashing. Keep the app running in development mode and then follow these steps:

1. Open **App.tsx** and change the referenced CSS class from **App** to **container** on the **div** element:

```
<div className="container">
```

```
<Alert ...>
```

```
...
```

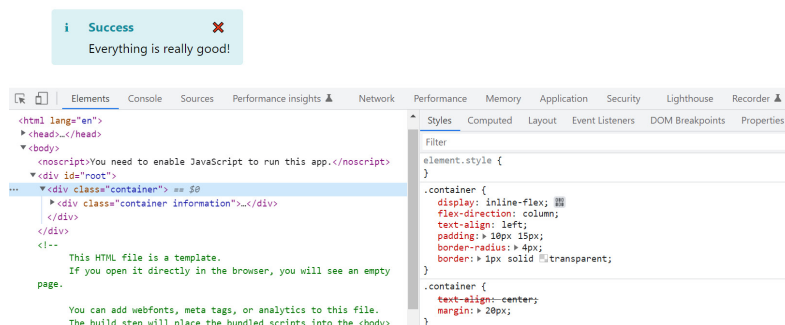
```
</Alert>
```

```
</div>
```

2. Open **App.css** and rename the **App** CSS class to **container** and also add **20px** of padding to it:

```
.container {  
  
    text-align: center;  
  
  
    padding: 20px;  
  
}
```

3. Now, look at the running app and notice that the alert is no longer centered horizontally on the page. Inspect the elements using the browser DevTools. If you inspect the **div** element from the **App** component, you will see that styles from the **container** CSS class in the alert component have been applied to it as well as the **container** CSS class we just added. So, the **text-align** CSS property is **left** rather than **center**.



BEM stands for **Block, Element, Modifier** and is a popular naming convention for CSS class names. More information can be found at the following link: <https://css-tricks.com/bem-101/>.

Here's a quick recap of this section:

- Create React App configures webpack to process CSS so that CSS files can be imported into React component files
- All the styles in an imported CSS file are applied to the app – there is no scoping or removing redundant styles

Next, we will learn about a styling approach that doesn't suffer from CSS clashes across components.

Using CSS modules

In this section, we will learn about an approach to styling React apps called **CSS modules**. We will start by understanding CSS modules and then we will use them within the alert component that we have been working on.

Understanding CSS modules

CSS modules is an open source library available on GitHub at <https://github.com/css-modules/css-modules>, which can be added to the webpack process to facilitate the automatic scoping of CSS class names.

A CSS module is a CSS file, just like in the previous section; however, the filename has an extension of **.module.css** rather than **.css**. This special extension allows webpack to differentiate a CSS module file from a plain CSS file so that it can be processed differently.

A CSS module file is imported into a React component file as follows:

```
import styles from './styles.module.css';
```

This is similar to the syntax of importing plain CSS, but a variable is defined to hold CSS class name mapping information. In the preceding code snippet, the CSS class name information is imported into a variable called **styles**, but the variable name can be anything we choose.

The CSS class name mapping information variable is an object containing property names corresponding to the CSS class names. Each class name property contains a value of a scoped class name to be used on a React component. Here is an example of the mapping object that has been imported into a component called **MyComponent**:

```
{
  container: "MyComponent_container__M7tzC",
  error: "MyComponent_error__vj80j"
}
```

The scope CSS class name starts with the component filename, then the original CSS class name, followed by a random string. This naming construct prevents class names from clashing.

Styles within a CSS module are referenced in a component's **className** attribute as follows:

```
<span className={styles.error}>A bad error</span>
```

The CSS class name on the element would then resolve to the scoped class name. In the preceding code snippets, **styles.error** would resolve to **MyComponent_error__ vj80j**. So, the styles in the running app will be the scoped style names and not the original class names.

Projects created using Create React App already have CSS modules installed and configured with webpack. This means we don't have to install CSS modules in order to start using them in our project.

Next, we will use CSS modules in the alert component we have worked on.

Using CSS modules in the alert component

Now that we understand CSS modules, let's use them in the alert component. Carry out the following steps:

1. Start by renaming **Alert.css** to **Alert.module.css**; this file can now be used as a CSS module.
2. Open **Alert.module.css** and change the CSS class names to camel case rather than kebab case. This will allow us to reference the scoped CSS class names more easily in the component – for example, **styles.headerText** rather than **styles["header-text"]**. The changes are as follows:

```
...
```

```
.headerIcon {
```

```
...
```

```
}
```

```
.headerText {
```

```
...
```

```
}
```

```
.closeButton {
```

```
...
```

```
}
```

3. Now, open **Alert.tsx** and change the CSS import statement to import the CSS module as follows:

```
import styles from './Alert.module.css';
```

4. In the JSX, change the class name references to use the scoped names from the CSS module:

```
<div className={` ${styles.container} ${styles[type]} `}>
```

```
<div className={styles.header}>
```

```
<span
```

```
...
```

```
className={styles.headerIcon}
```

```
>
```

```
{type === "warning" ? "⚠" : "i"}
```

```
</span>
```

```

    {heading && (

        <span className={styles.headerText}>{heading}</span>

    )}

    {closable && (

        <button

            ...

            className={styles.closeButton}

        >

            ...

        </button>

    )}

</div>

<div className={styles.content}>{children}</div>

</div>

```

5. Start the app by running **npm start** in the terminal.

After a few seconds, the styled alert will appear. This time the alert will be horizontally centered, which is a sign that styles are no longer clashing.

6. Inspect the elements in the DOM using the browser's DevTools. You will see that the alert component is now using scoped CSS class names. This means the alert container styles no longer clash with the app container styles.

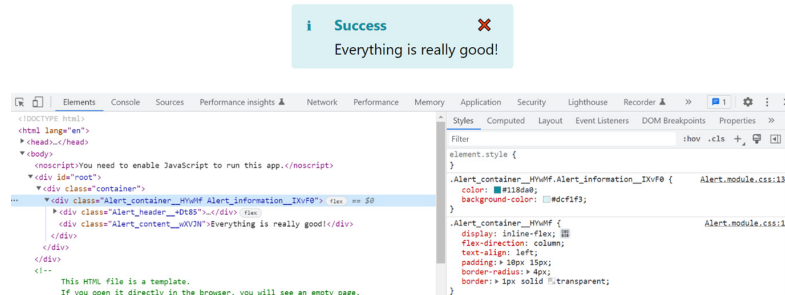


Figure 5.5 – The CSS module scoped class names

7. Stop the running app before continuing by pressing *Ctrl* + *C*.
8. To round off our understanding of CSS modules, let's see what happens to the CSS in a production build. However, before we do that, let's add a redundant CSS class at the bottom of **Alert.module.css**:

```
...

.content {

    margin-left: 30px;

    color: #000;

}

.redundant {

    color: red;

}
```

9. Now create a production build by executing `npm run build` in the terminal.

After a few seconds, the build artifacts are created in the `build` folder.

10. Open the bundled CSS file and you will notice the following points:

1. It contains all the CSS from `index.css`, `App.css`, and the CSS module we just created.
2. The class names from the CSS module are scoped. This will ensure that the styles in production don't clash, as they did not in development mode.
3. It contains the redundant CSS class name from the CSS module.



```
# main.4e8b03b1.css X
build > static > css > # main.4e8b03b1.css > ...
1  :__YqM9b{color:■#000;margin-left:30px}.Alert_redundant__qwosS{color:■red}
2
```

Figure 5.6 – The redundant CSS class included in CSS bundle

That completes the refactoring of the alert component to use CSS modules.

NOTE

For more information on CSS modules, visit the *GitHub repository* at <https://github.com/css-modules/css-modules>.

Here's a recap of what we have learned about CSS modules:

- CSS modules allow CSS class names to be automatically scoped to a React component. This prevents styles for different React components from clashing.
- CSS modules isn't a standard browser feature; instead, it is an open source library that can be added to the webpack process.
- CSS modules are pre-installed and pre-configured in projects created with Create React App.
- Similar to plain CSS, redundant CSS classes are not pruned from the production CSS bundle.

Next, we will learn about another approach to styling React apps.

Using CSS-in-JS

In this section, we start by understanding CSS-in-JS and its benefits. We will then refactor the alert component we have used to implement CSS-in-JS and observe how it differs from CSS modules.

Understanding CSS-in-JS

CSS-in-JS isn't a browser feature, and it isn't even a specific library – instead, it is a type of library. Popular examples of CSS-in-JS libraries are **styled-components** and **Emotion**. There isn't a significant difference between styled-components and Emotion – they are both popular and have similar APIs. We will use Emotion in this part.

Emotion generates styles that are scoped, similar to CSS modules. However, you write the CSS in JavaScript rather than in a CSS file – hence the name *CSS-in-JS*. In fact, you can write the CSS directly on JSX elements as follows:

```
<span
  css={css`
    font-weight: 700;
    font-size: 14;
  `}
>
  {text}
</span>
```

Each CSS-in-JS library's syntax is slightly different – the preceding example is a code snippet from Emotion styling.

Having styles directly on the component allows a developer to fully understand the component without having to visit another file. This obviously increases the file size, which can make the code harder to read. However, child components can be identified and extracted out of the file to mitigate large file sizes. Alternatively, styles can be extracted from component files into a JavaScript function that is imported.

A massive benefit of CSS-in-JS is that you can mix logic into the style, which is really useful for highly interactive apps. The following example contains a conditional **font-weight** dependent on an **important** prop and a conditional **font-size** dependent on a **mobile** prop:

```
<span
  css={css`
    font-weight: ${important ? 700 : 400};
    font-size: ${mobile ? 15 : 14};
  `}
>
  {text}
</span>
```

JavaScript string interpolation is used to define the conditional statement.

The equivalent plain CSS would be similar to the following example, with separate CSS classes created for the different conditions:

```
<span
  className={`${important ? "text-important" : ""} ${
    mobile ? "text-important" : ""
 }`}
>
  {text}
</span>
```

If a style on an element is highly conditional, then CSS-in-JS is arguably easier to read and certainly easier to write.

Next, we will use Emotion in the alert component we have worked on.

Using Emotion in the alert component

Now that we understand CSS-in-JS, let's use Emotion in the alert component.at :

1. Create React App doesn't install and set up Emotion, so we first need to install Emotion. Run the following command in the terminal:

```
npm i @emotion/react
```

This will take a few seconds to install.

2. Open **Alert.tsx** and remove the CSS module import.
3. Add an import for the **css** prop from Emotion with a special comment at the top of the file:

```
/** @jsxImportSource @emotion/react */
```

```
import { css } from '@emotion/react';
```

```
import { useState } from 'react';
```

This special comment changes JSX elements to be transpiled using Emotion's **jsx** function instead of React's **createElement** function. Emotion's **jsx** function adds styles to elements containing Emotion's **css** prop.

4. In the JSX, we need to replace all the **className** props with the equivalent Emotion **css** attributes. The styles are largely the same as defined in the CSS file we created earlier, so the explanations won't be repeated.

We will style one element at a time, starting with the outer **div** element:

```
<div
  css={css`
    display: inline-flex;
    flex-direction: column;
    text-align: left;
    padding: 10px 15px;
    border-radius: 4px;
    border: 1px solid transparent;
    color: ${type === "warning" ? "#e7650f" : "#118da0"};
    background-color: ${type === "warning"
```

```

      ? "#f3e8da"
      : "#dcf1f3"};
    `}
  >
    ...
</div>

```

There are a few important points to explain in this code snippet:

- The **css** attribute isn't usually valid on JSX elements. The special comment at the top of the file (`/** @jsxImportSource @emotion/react */`) allows this.
- The **css** attribute is set to a **tagged template literal**. This is a special string that gets processed by the function specified before it, which is a function called **css** in this case. For more information on tagged template literals, see https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Template_literals.
- The tagged template literal converts the style to a CSS class at runtime. We will verify this in *step 14*.
- String interpolation is used to implement the conditional styles for the colors. Remember that we had to define three CSS classes using plain CSS or CSS modules. This CSS-in-JS version is arguably more readable and certainly more concise.

5. Next, style the header container:

```

<div

  css={css`

    display: flex;

    align-items: center;

    margin-bottom: 5px;

  `}

>

  <span role="img" ... > ... </span>

```

```
<span ...>{heading}</span>
```

```
{closable && ...}
```

```
</div>
```

6. Next, style the icon as follows:

```
<span
```

```
role="img"
```

```
aria-label={type === "warning" ? "Warning" : "Information"}
```

```
css={css`
```

```
width: 30px;
```

```
`}
```

```
>
```

```
{type === "warning" ? "⚠" : "i"}
```

```
</span>
```

7. Then, style the heading as follows:

```
<span
```

```
css={css`
```

```
font-weight: bold;
```

```
`}
```

```
>
```

```
{heading}
```

```
</span>
```

8. Now, style the close button as follows:

```
{closable && (
```

```
<button
```

```
...
```

```
css={css`
```

```
border: none;
```

```
background: transparent;
```

```
margin-left: auto;
```

```
cursor: pointer;
```

```
`}
```

```
>
```

```
...
```

```
</button>
```

```
})
```

9. Finally, style the message container as follows:

```
<div
```

```
css={css`
```

```
margin-left: 30px;
```

```
color: #000;
```

```
`}
```

```
>
```

```
{children}
```

```
</div>
```

10. Run the app by running **npm start** in the terminal. The alert component will appear like it was before.
11. Inspect the elements in the DOM using the browser's DevTools. The alert component uses scoped CSS class names, similar to CSS modules:

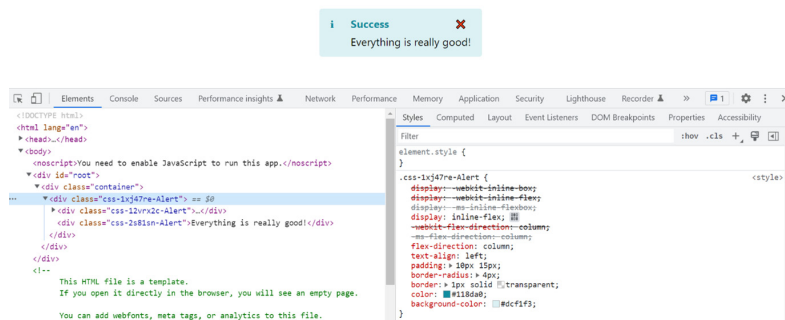


Figure 5.7 – Emotion’s scoped class names

12. Stop the running app before continuing by pressing **Ctrl + C**.
13. To round off our understanding of Emotion, let’s see what happens to the CSS in a production build. First, create a production build by executing **npm run build** in the terminal.

After a few seconds, the build artifacts are created in the **build** folder.

14. Open the bundled CSS file from the **build/static/css** folder. Notice that the Emotion styles are not there. This is because Emotion generates the styles at runtime via JavaScript rather than at build time. If you think about it, the styles can’t be generated at build time because they may depend on JavaScript variables whose values are only known at runtime.

This completes the refactoring of the alert component to use CSS-in-JS.

NOTE

For more information on emotion, visit their website at <https://emotion.sh/docs/introduction>.

Here’s a recap of what we learned about Emotion and CSS-in-JS:

- Styles for a CSS-in-JS library are defined in JavaScript rather than a CSS file.
- Emotion’s styles can be defined directly on a JSX element using a **css** attribute.
- A huge benefit is that conditional logic can be added directly to the styles, which helps us style interactive components more quickly.
- Emotion styles are applied at runtime rather than at build time because they depend on JavaScript variables. While this allows conditional styling logic to be elegantly defined, it does mean a small performance penalty because the styles are created and applied at runtime.

Next, we will learn about another different approach to styling React frontends.

Using Tailwind CSS

In this section, we will start by understanding Tailwind CSS and its benefits. We will then refactor the alert component we have been using to use Tailwind and observe how it differs from other approaches we have tried.

Understanding Tailwind CSS

Tailwind is a set of prebuilt CSS classes that can be used to style an app. It is referred to as a **utility-first CSS framework** because the prebuilt classes can be thought of as flexible utilities.

An example CSS class is **bg-white**, which styles the background of an element white – *bg* is short for *background*. Another example is **bg-orange-500**, which sets the background color to a 500 shade of orange. Tailwind contains a nice color palette that can be customized.

The utility classes can be used together to style an element. The following example styles a button element in JSX:

```
<button className="border-none rounded-md bg-emerald-700 text-white cursor-pointer">
  ...
</button>
```

Here's an explanation of the classes used in the preceding example:

- **border-none** removes the border of an element.
- **rounded-md** rounds the corners of an element border. The *md* stands for *medium*. Alternatively, **lg** (large) could have been used or even **full**, for more rounded borders.
- **bg-emerald-700** sets the element background color to a 700 shade of emerald.
- **text-white** sets the element text color to white.
- **cursor-pointer** sets the element cursor to a pointer.

The utility classes are low-level and focused on styling a very specific thing. This makes the classes flexible, allowing them to be highly reusable.

Tailwind can specify that a class should be applied when the element is in a hover state by prefixing it with **hover:**. The following example sets the button background to a darker shade of emerald when hovered:

```
<button className="md border-none rounded-md bg-emerald-700 text-white cursor-pointer hover
  ...
</button>
```

So, a key point of Tailwind is that we don't write new CSS classes for each element we want to style – instead, we use a large range of well-thought-through existing classes. A benefit of this approach is that it helps an app look nice and consistent.

NOTE

For more information on Tailwind, refer to their website at the following link: <https://tailwindcss.com/>. The Tailwind website is a crucial resource

for searching and understanding all the different utility classes that are available.

Next, we will install and configure Tailwind in the project containing the alert component we have been working on.

Installing and configuring Tailwind CSS

Now that we understand Tailwind, let's install and configure it in the alert component project. To do this, carry out the following steps:

1. In the Visual Studio project, start by installing Tailwind by running the following command in a terminal:

```
npm i -D tailwindcss
```

The Tailwind library is installed as a development dependency because it's not required at runtime.

2. Tailwind integrates into Create React App projects using a library called **PostCSS**. PostCSS is a tool that transforms CSS using JavaScript and Tailwind runs as a plugin in it. Install PostCSS by running the following command in the terminal:

```
npm i -D postcss
```

3. Tailwind also recommends another PostCSS called **Autoprefixer**, which adds vendor prefixes to CSS. Install this by running the following command in the terminal:

```
npm i -D autoprefixer
```

4. Next, run the following command in a terminal to generate configuration files for Tailwind and PostCSS:

```
npx tailwindcss init -p
```

After a few seconds, the two configuration files are created. The Tailwind configuration file is called **tailwind.config.js**, and the PostCSS configuration file is called **postcss.config.js**.

5. Open **tailwind.config.js** and specify the path to the React components as follows:

```
module.exports = {
```

```
  content: [
```

```

      './src/**/*.{js,jsx,ts,tsx}'

    ],

    theme: {

      extend: {},

    },

    plugins: [],

  }

```

6. Now, open **index.css** in the **src** folder and add the following three lines at the top of the file:

```

@tailwind base;

@tailwind components;

@tailwind utilities;

```

These are called **directives** and will generate the CSS required by Tailwind during the build process.

Tailwind is now installed and ready to use.

Next, we will use Tailwind to style the alert component we have been working on.

Using Tailwind CSS

Now, let's use Tailwind to style the alert component. We will remove emotion's **css** JSX attribute and replace it with Tailwind utility class names in the JSX **className** attribute. To do this, carry out the following steps:

1. Open **Alert.tsx** and start by removing the special emotion comment and the **css** import statement from the top of the file.
2. Replace the **css** attribute with a **className** attribute on the outermost **div** element as follows:

```
<div  
  
  
  className={`inline-flex flex-col text-left px-4 py-3 rounded-md border-1 border-tr  
  
  >  
  
  ...  
  
</div>
```

Here is an explanation of the utility classes that were just used:

- **inline-flex** and **flex-col** create an inline flexbox that flows vertically
- **text-left** aligns items to the left
- **px-4** adds 4 spacing units of left and right padding
- **py-3** adds 3 spacing units of top and bottom padding
- We have encountered **rounded-md** before – this rounds the corners of the **div** element
- **border-1** and **border-transparent** add a transparent 1 px border

NOTE

Spacing units are defined in Tailwind and are a proportional scale. One spacing unit is equal to 0.25rem, which translates roughly to 4px.

3. Still on the outermost **div** element, add the following conditional styles using string interpolation:

```
<div  
  
  
  className={`inline-flex flex-col text-left px-4 py-3 rounded-md border-1 border-transp  
  
  
  type === 'warning' ? 'text-amber-900' : 'text-teal-900'
```

```

    } ${type === 'warning' ? 'bg-amber-50' : 'bg-teal-50'}}`}}

    >

    ...

</div>

```

The text color is set to a 900 amber shade for warning alerts and a 900 teal shade for information alerts. The background color is set to a 50 amber shade for warning alerts and a 50 teal shade for information alerts.

4. Next, replace the **css** attribute with a **className** attribute on the header container as follows:

```

<div className="flex items-center mb-1">

    <span role="img" ... > ... </span>

    <span ... >{heading}</span>

    {closable && ...}

</div>

```

Here is an explanation of the utility classes that were just used:

- **flex** and **items-center** create a horizontal flowing flexbox where the items are centered vertically
- **mb-1** adds a 1 spacing unit margin at the bottom of the element

5. Replace the **css** attribute with a **className** attribute on the icon as follows:

```

<span role="img" ... className="w-7">

    {type === 'warning' ? '△' : 'i'}

```

```
</span>
```

w-7 sets the element to a width of 7 spacing units.

6. Replace the **css** attribute with a **className** attribute on the heading as follows:

```
<span className="font-bold">{heading}</span>
```

font-bold sets the font weight to be bold on the element.

7. Replace the **css** attribute with a **className** attribute on the close button as follows:

```
{closable && (
```

```
<button
```

```
...
```

```
className="border-none bg-transparent ml-auto cursor-pointer"
```

```
>
```

```
...
```

```
</button>
```

```
)}
```

Here, **border-none** removes the element border, and **bg-transparent** makes the element background transparent. **ml-auto** sets the left margin to automatic, which right aligns the element. **cursor-pointer** sets the mouse cursor to a pointer.

8. Finally, replace the **css** attribute with a **className** attribute on the message container as follows:

```
<div className="m1-7 text-black">
```

```
{children}
```

```
</div>
```

m1-7 sets the left margin on the element to 7 spacing units and **text-black** sets the text color to black.

9. Run the app by running **npm start** in the terminal. After a few seconds, the app will appear in the browser.

Notice that the alert component looks a bit nicer because of Tailwind's default color palette and consistent spacing.

10. Inspect the elements in the DOM using the browser's DevTools. Notice the Tailwind utility classes being used and notice the spacing units use CSS rem units.

A key point to notice is that no CSS class name scoping occurs. There is no need for any scoping because the classes are general and reusable and not specific to any element.

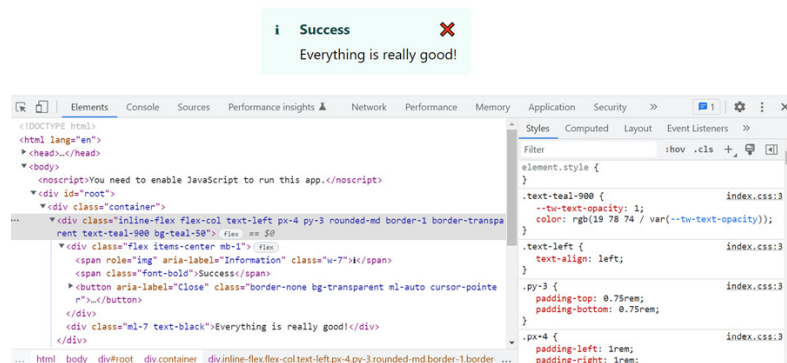


Figure 5.8 – A styled alert using Tailwind

11. Stop the running app before continuing by pressing **Ctrl + C**.
12. To round off our understanding of Tailwind, let's see what happens to the CSS in a production build. First, create a production build by executing **npm run build** in the terminal.

After a few seconds, the build artifacts are created in the **build** folder.

13. Open the bundled CSS file from the **build/static/css** folder. Notice the base Tailwind styles at the start of the file. You will also see that all the Tailwind classes that we used are in this file.

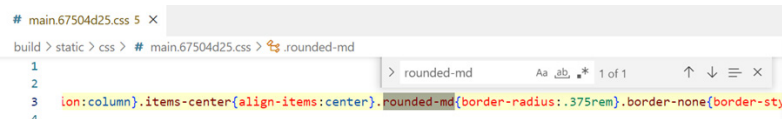


Figure 5.9 – Tailwind CSS classes in a bundled CSS file

NOTE

An important point is that Tailwind doesn't add all its CSS classes – that would produce a massive CSS file! Instead, it only adds the CSS classes used in the app.

That completes the process of refactoring the alert component to use Tailwind.

Here's a recap of what we learned about Tailwind:

- Tailwind is a well-thought-through collection of reusable CSS classes that can be applied to React elements
- Tailwind has a nice default color palette and a 4 px spacing scale, both of which can be customized
- Tailwind is a plugin for PostCSS and executed at build time
- Tailwind does not incur a runtime performance penalty like Emotion, because the styles aren't created and applied at runtime
- Only classes used on React elements are included in the CSS build bundle

Next, we will make the icons in the alert component look a bit nicer.

Using SVGs

In this section, we will learn how to use SVG files in React and how to use them for the icons in the alert component.

Understanding how to use SVGs in React

SVG stands for **Scalable Vector Graphics** and it is made up of points, lines, curves, and shapes based on mathematical formulas rather than specific pixels. This allows them to scale when resized without distortion. The quality of icons is important to get right – if they are distorted, they make the whole app feel unprofessional. Using SVGs for icons is common in modern web development.

Create React App configures webpack to use SVG files when a project is created. In fact, **logo.svg** is referenced in the template **App** component as follows:

```
import logo from './logo.svg';
...
function App() {
  return (
    <div className="App">
      <header className="App-header">
        <img src={logo} className="App-logo" alt="logo" />
        ...
      </header>
    </div>
  );
}
```

```

    }
    export default App;

```

In the preceding example, **logo** is imported as a path to the SVG file, which is then used on the **src** attribute on the **img** element to display the SVG.

An alternate way of referencing SVGs is to reference them as a component as follows:

```

import { ReactComponent as Logo } from './logo.svg';
function SomeComponent() {
  return (
    <div>
      <Logo />
    </div>
  );
}

```

SVG React components are available in a named import called **ReactComponent**. In the preceding example, the SVG component is aliased with the name **Logo**, which is then used in the JSX.

Next, we will learn how to use SVGs in the alert component.

Adding SVGs to the alert component

Carry out the following steps to replace the emoji icons in the alert component with SVGs:

1. First, create three files called **cross.svg**, **info.svg**, and **warning.svg** in the **src** folder. Then, copy and paste the content of these from the code file s shared at .
2. Open **Alert.tsx** and add the following import statements to import the SVGs as React components:

```

import { ReactComponent as CrossIcon } from './cross.svg';

import { ReactComponent as InfoIcon } from './info.svg';

import { ReactComponent as WarningIcon } from './warning.svg';

```

We have given the SVG components appropriately named aliases.

3. Update the **span** element containing the emoji icons to use SVG icon components as follows:

```

<span

```



```

    role="img"

    aria-label={type === 'warning' ? 'Warning' : 'Information'}

    className="inline-block w-7"

  >

    {type === 'warning' ? (

      <WarningIcon className="fill-amber-900 w-5 h-5" />

    ) : (

      <InfoIcon className="fill-teal-900 w-5 h-5" />

    )}

  </span>;

```

We have used Tailwind to size and color the icons appropriately.

4. Next, update the emoji close icon to the SVG close icon as follows:

```

<button

  aria-label="Close"

  onClick={handleCloseClick}

  className="border-none bg-transparent ml-auto cursor-pointer"

```

>

```
<CrossIcon />
```

```
</button>
```

5. Run the app by running **npm start** in the terminal. After a few seconds, the app will appear in a browser containing the improved alert component:

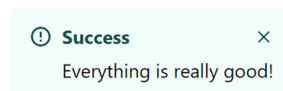


Figure 5.10 – An alert with an SVG icon

That completes the alert component – it is looking much better now.

Here's a quick recap of what we learned about using SVGs in React apps:

- Webpack needs to be configured to bundle SVG files and Create React App does this configuration for us
- The default import for an SVG file is the path to the SVG, which can then be used in an **img** element
- A named import called **ReactComponent** can be used to reference the SVG as a React component in JSX

Next, we will summarize what we have learned in this part.

Summary

In this part, we learned about four methods of styling.

First, we learned that plain CSS could be used to style React apps, but all the styles in the imported CSS file are bundled regardless of whether a style is used. Also, the styles are not scoped to a specific component – we observed the **container** CSS class names clashing with the **App** and **Alert** components.

Next, we learned about CSS modules, which allows us to write plain CSS files imported in a way that scopes styles to the component. We learned that CSS modules is an open source library pre-installed and pre-configured in projects created with Create React App. We saw how this resolved the CSS clashing problem but didn't remove redundant styles.

Then, we discussed CSS-in-JS libraries, which allow styles to be defined directly on the React component. We used emotion's **css** prop to style the alert component without an external CSS file. The nice thing about this approach is that conditional-style logic can be implemented more quickly. We learned that emotion's styles are scoped like CSS modules, but the scoping happens at runtime rather than at build time. We also under-

stood that the small performance cost of this approach is because of the styles being created at runtime.

The fourth styling approach we looked at was using Tailwind CSS. We learned that Tailwind provides a set of reusable CSS classes that can be applied to React elements, including a nice default color palette and a 4 px spacing scale, both of which can be customized. We learned that only the used Tailwind classes are included in the production build.

Finally, we learned that Create React App configures webpack to enable the use of SVG files. SVGs can be referenced as a path in an **img** element or as a React component using a **ReactComponent** named **import**.

In the next part, we will look at implementing multiple pages in React apps with a popular library called React Router.

Questions

Answer the following questions to check what you have learned about React styling:

1. Why could the following use of plain CSS be problematic?

```
<div className="wrapper"></div>
```

2. We have a component styled using CSS modules as follows:

```
import styles from './styles1.module.css';

function ComponentOne() {

  return <div className={styles.wrapper}></div>;

}
```

We have another component styled using CSS modules as follows:

```
import styles from './styles2.module.css';
function ComponentTwo() {
  return <div className={styles.wrapper}></div>;
}
```

Will the styles of these **div** elements clash, given that they are using the **wrapper** class name?

3. We have a component styled using CSS modules as follows:

```
import styles from './styles3.module.css';

function ComponentThree() {

  return <div className={styles.wrapper}>

    </div>

  }
}
```

The styles in **styles3.module.css** are as follows:

```
.wrap {
  display: flex;
  align-items: center;
  background: #e7650f;
}
```

The styles aren't being applied when the app is run. What is the problem?

4. We are defining a reusable button component with a **kind** prop that can be **"square"** or **"rounded"**. The rounded button should have a 04 px border radius, and the square button should have no border radius. How could we define this conditional style using Emotion's **css** prop?
5. We are styling a button element using Tailwind. It is currently styled as follows:

```
<button className="bg-blue-500 text-white font-bold py-2 px-4 rounded">

  Button

</button>
```

How can we enhance the style by making the button background a 700 shade of blue when the user hovers over it?

6. A logo SVG is referenced as follows:

```
import Logo from './logo.svg';
```

```
function LogoComponent() {
```

```
  return <Logo />;
```

```
}
```

However, the logo isn't rendered. What is the problem?

7. We are styling a button element using Tailwind that has a **color** prop to determine its color and is styled as follows:

```
<button className={`bg-${color}-500 text-white font-bold py-2 px-4 rounded`}>
```

```
  Button
```

```
</button>
```

However, the button color doesn't work. What is the problem?

Answers

1. The wrapper CSS class could clash with other classes. To reduce this risk, the class name can be manually scoped to the component:

```
<div className="card-wrapper"></div>
```

2. The CSS won't clash because CSS modules will scope the class names to each component.
3. The wrong class name is referenced in the component – it should be **wrap** rather than **wrapper**:

```
import styles from './styles3.module.css';
```

```
function ComponentThree() {
```

```
  return <div className={styles.wrap}>
```

```
</div>
```

```
}
```

4. The **css** prop on the button could be as follows:

```
<button
```

```
  css={css`
```

```
    border-radius: ${kind === "rounded" ? "4px" : "0px"};
```

```
`}
```

```
>
```

```
  ...
```

```
</button>
```

5. The style can be adjusted as follows to include the hover style:

```
<button className="bg-blue-500 hover:bg-blue-700 text-white font-bold py-2 px-4 rounded"
```

```
  ...
```

```
</button>
```

6. **Logo** will hold the path to the SVG rather than a component. The import statement can be adjusted as follows to import a logo component:

```
import { ReactComponent as Logo } from './logo.svg';
```

```
function LogoComponent() {
```

```
return <Logo />;
```

```
}
```

7. The `bg-${color}-500` class name is problematic because this can only be resolved at runtime because of the `color` variable. The used Tailwind classes are determined at build time and added to the bundle, meaning the relevant background color classes won't be bundled. This means that the background color style won't be applied to the button.